

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 05 | Course: PARS Lab | Framework: Python 3.14 / Flask / NLTK | Repository: [dSaxmonis/pars-lab](https://github.com/dSaxmonis/pars-lab)

Practical Name

Text Paragraph Segmentation, Stop Words Removal, Stemming, Lemmatization, and Bayesian Classification Using Flask

AIM

To prepare an interactive web application using Flask in Python to extract a text paragraph, separate it into individual sentences, filter out English stop words, apply stemming and lemmatization, and classify the processed text into domain categories using a Multinomial Naive Bayes classifier.

THEORY

1. Sentence Segmentation & Tokenization

Sentence segmentation separates continuous text into distinct grammatical sentences. Word tokenization further isolates individual lexical tokens while removing punctuation marks.

2. Stop Words Filtering

Stop words represent ubiquitous syntactic connectors (e.g., *the, is, in, at, which, for*) that lack domain discriminating information. Filtering reduces noise and computational complexity:

```
T_filtered = { w ∈ T | w ∉ W_stop }
```

3. Stemming vs. Lemmatization

- **Porter Stemmer (Stemming):** Employs heuristic suffix-stripping rules to reduce inflectional forms (e.g., *algorithms* → *algorithm*, *computing* → *comput*).
- **WordNet Lemmatizer (Lemmatization):** Utilizes full morphological vocabulary parsing to return canonical dictionary headwords (lemmas).

4. Multinomial Naive Bayes Mathematical Formulas

1. Bayes Rule for Documents:

$$P(c | d) = [P(c) \times \prod P(w_j | c)] / P(d)$$

2. Class Prior Probability:

$$P(c) = N_c / N$$

3. Laplace (Add-1) Smoothed Word Likelihood:

$$P(w_j | c) = (\text{count}(w_j, c) + 1) / (\sum \text{count}(w, c) + |V|)$$

4. MAP Decision Rule (Log-Space):

$$c_{\text{hat}} = \text{argmax}_c [\ln P(c) + \sum \ln P(w_j | c)]$$

5. Normalized Posterior Probability (Softmax):

$$P(c | d) = \exp(\ln P(c | d)) / \sum [\exp(\ln P(c' | d))]$$

STEPS & EXECUTION

Step 1: Environment Setup & NLTK Resource Verification

Configured `exp 5/` directory and validated NLTK packages (punkt, punkt_tab, stopwords, wordnet).

Step 2: Sentence Segmentation & Word Tokenization

Parsed paragraph using `sent_tokenize()` and `word_tokenize()` to establish lexical boundary units.

Step 3: Stop Words Filtering, Stemming & Lemmatization

Applied stop word exclusion, Porter Stemmer, and WordNet Lemmatizer across all extracted tokens.

Step 4: Interactive Web Server Launch on Port 5007

Launched interactive Flask web server on <http://127.0.0.1:5007> for real-time text analysis and Bayesian inference.

OUTPUT / DASHBOARD RESULTS

Output 1: Paragraph Input Form & Presets

Web interface accepting custom text paragraphs and benchmark testing presets.

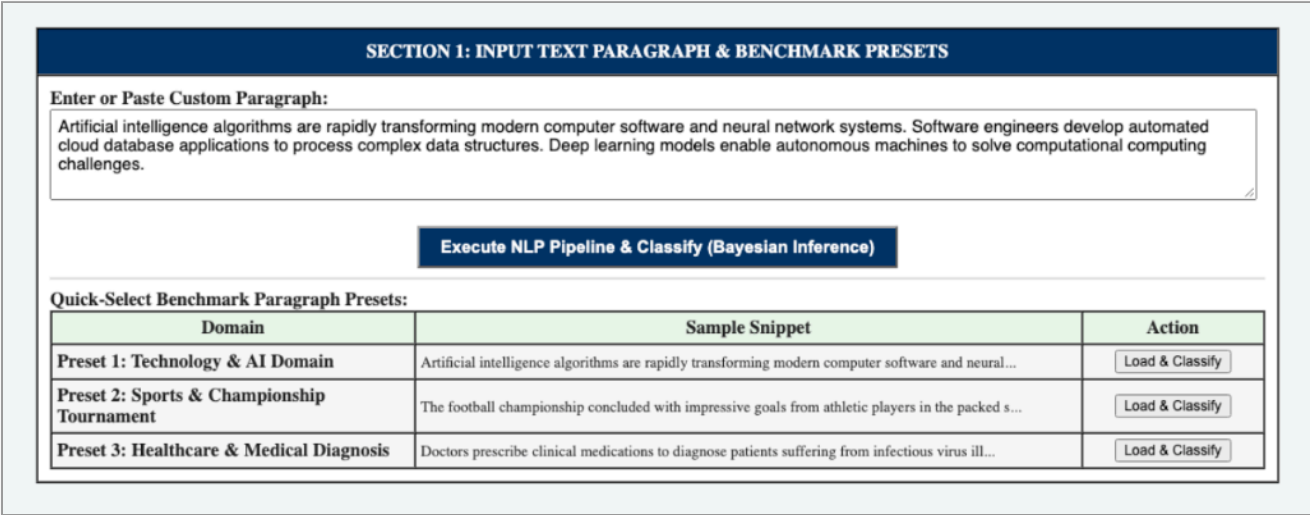


Figure 5.1: Section 1 — Interactive text input form and quick benchmark selections.

Output 2: Sentence-by-Sentence NLP Preprocessing Pipeline Table

Complete transformation breakdown showing original sentences, filtered stop words, stemmed tokens, and lemmatized lemmas.

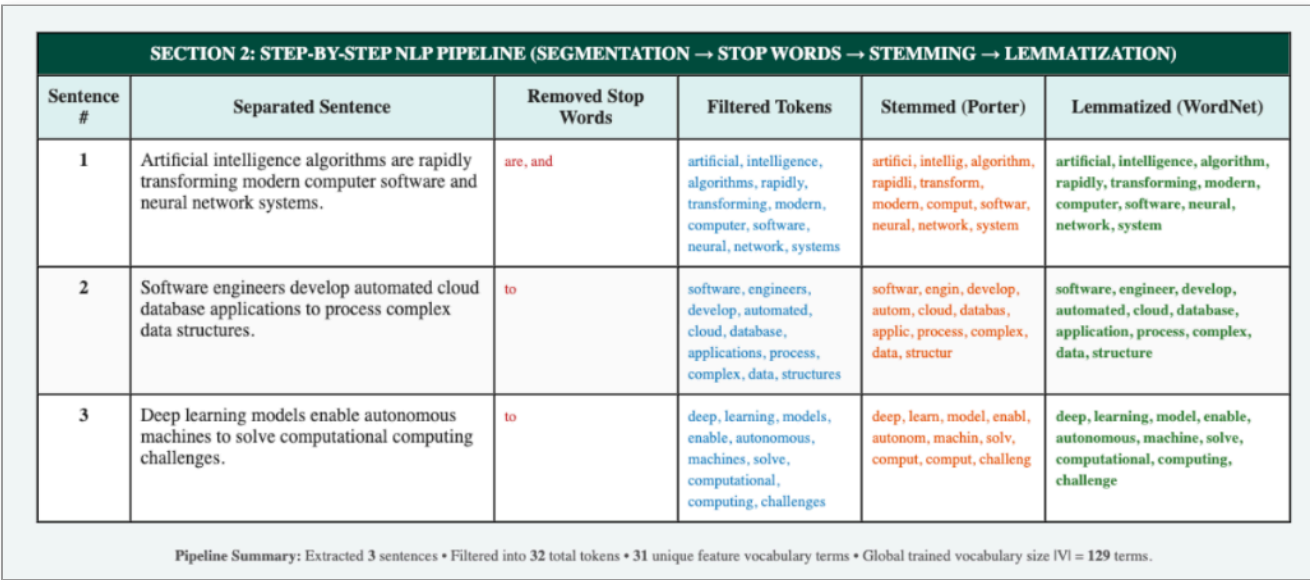


Figure 5.2: Section 2 — Step-by-step lexical preprocessing table per sentence.

Output 3: Multinomial Naive Bayes Classification Results

Calculated prior probabilities, log-likelihood sums, normalized posteriors, and MAP winner identification.

SECTION 3: MULTINOMIAL NAIVE BAYES TEXT CLASSIFICATION				
Category / Domain (c)	Class Prior P(c)	Log-Likelihood Score $\sum \log P(w c)$	Posterior Probability P(c d)	Confidence / Status
Technology	0.3333	-148.2644	1.0	★ Predicted Winner (100.0%)
Sports	0.3333	-167.2732	0.0	0.0%
Health / Medical	0.3333	-167.9766	0.0	0.0%
Final Bayesian Decision: Categorized as Technology with 100.00% confidence.				

Figure 5.3: Section 3 — Bayesian text classification probability results table.

Output 4: Bayesian Posterior Distribution & Salient Keywords Plot

Visualizing posterior probabilities alongside top informative keywords extracted from the test paragraph.

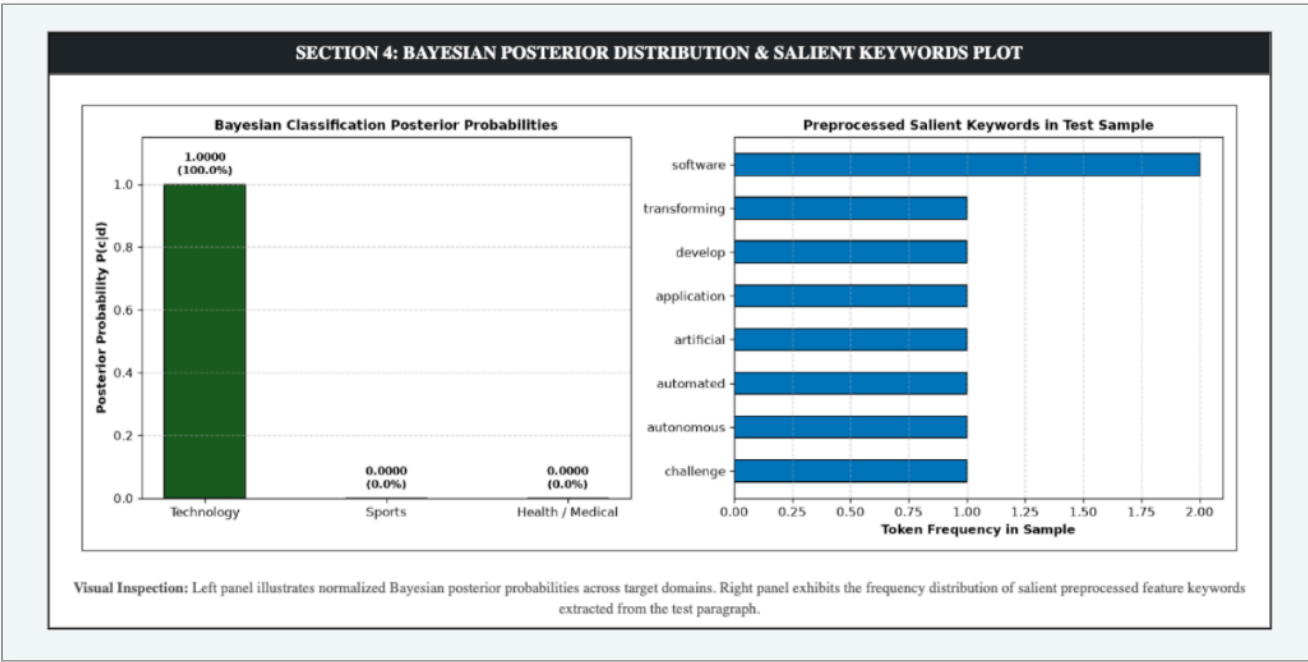


Figure 5.4: Section 4 — Posterior probability bar chart and salient keyword frequencies.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSAxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/requirements.txt
- **Lab Manual (Markdown):**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_5/LAB_PRACTICAL_FILE.md

CONCLUSION

In this experiment, an end-to-end natural language processing and text categorization pipeline was successfully implemented using Flask and NLTK in Python. The workflow demonstrated sentence boundary segmentation, token cleaning, stop words removal, Porter stemming, and WordNet lemmatization. Finally, a Multinomial Naive Bayes classifier with Laplace smoothing accurately predicted the topical domain of test paragraphs based on word likelihoods and prior probabilities, displaying the entire transformation and probability distribution on an interactive web page.